

VisionEngine

उपयोगकर्ता की तरह देखें यूआई — कंप्यूटर विज्ञान के साथ-साथ LLM विज्ञान विश्लेषण और नेविगेशन के लिए।

सारांश

VisionEngine एक डिक्पल्ड Go टूलकिट है जो क्लासिक कंप्यूटर विज्ञान को LLM-आधारित विज्ञान के साथ जोड़कर यूजर इंटरफ़ेस का विश्लेषण करती है, यूआई तत्वों और दृश्य समस्याओं का पता लगाती है, और ऐप स्क्रीन ट्रांज़िशन के नेविगेशन ग्राफ़ बनाती है — जिसमें प्लग करने योग्य मल्टी-प्रोवाइडर विज्ञान बैकएंड और OpenCV को बिल्ड टैग के पीछे गेटेड किया गया है।

संक्षिप्त विवरण

यूआई विश्लेषण और नेविगेशन-ग्राफ़ निर्माण के लिए एक पुनःप्रयोगी Go मॉड्यूल। यह एक विश्लेषक परत (यूआई तत्व, स्क्रीन अंतर, दृश्य समस्याएँ), BFS पाथफाइंडिंग और DOT/JSON/मरमेड एक्सपोर्ट के साथ एक नेविगेशन ग्राफ़, तथा GPT-4o, Claude, Gemini, Qwen-VL और अन्य के लिए LLM-विज्ञान एडाप्टर प्रदान करता है।

विस्तृत विवरण

अधिकांश यूआई टेस्ट ऑटोमेशन वास्तव में अंधा होता है। यह एक्सेसिबिलिटी ट्री और DOM सिलेक्टरों पर निर्भर करता है — जो मशीन की नज़र में इंटरफ़ेस होता है — और वह सब छूट जाता है जो एक इंसान वास्तव में अनुभव करता है: क्या बटन दृश्यमान रूप से रेंडर हुआ है, क्या लेआउट टूट गया है, या वह स्क्रीन सही है जिसकी उसे उम्मीद थी।

VisionEngine इस अंतर को पाटता है, ऑटोमेशन को वास्तविक दृष्टि प्रदान करता है — यूआई को देखने और उसके बारे में इंसानी तरीके से सोचने की क्षमता। यह चार सहयोगी परतों में व्यवस्थित है जो कच्चे पिक्सेल से लेकर पूरे ऐप की समझ तक का निर्माण करती हैं।

विश्लेषक (एनालाइज़र) स्थिर अनुबंध को परिभाषित करता है — इंटरफ़ेस (Analyzer, VideoProcessor) और मूल्य प्रकार (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) एक StubAnalyzer संदर्भ कार्यान्वयन के साथ — ताकि उपभोक्ता तत्वों का पता लगा सकें, स्क्रीन की तुलना कर सकें और दृश्य समस्याओं को अनुबंध के तहत सामने ला सकें जो उनके लिए अस्थिर न हो। नेविगेशन ग्राफ़ एकल स्क्रीन के दृष्टिकोण को पूरे एप्लिकेशन तक ले जाता है, इसे स्क्रीन ट्रांज़िशन के निर्देशित ग्राफ़ के रूप में मॉडल करता है जिसमें BFS पाथफाइंडिंग और तीन एक्सपोर्ट बैकएंड (DOT, JSON, मरमेड) शामिल हैं, ताकि

ऑटोमेशन न केवल एक स्क्रीन देख सके बल्कि किसी भी अन्य तक पहुँचने का मार्ग भी तय कर सके — और यह स्ट्रेस, ऑटोमेशन, इंटीग्रेशन और सुरक्षा परीक्षण सूट के साथ आता है जो इसे साबित करते हैं।

LLM विज्ञान परत आधुनिक मल्टीमॉडल तर्क जोड़ती है: एक VisionProvider इंटरफ़ेस जिसमें OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, किमी, स्टेपGUI, एस्टिका और Ollama के लिए एडाप्टर होते हैं, जिन्हें FallbackChain के माध्यम से संयोजित किया जाता है ताकि कोई असफल, रेट-लिमिटेड या कमजोर प्रोवाइडर अगले पर ग्रेसफुली डिग्रेड हो जाए बजाय कि पूरे रन को प्रभावित करने के। कॉन्फ़िगरेशन परत env-वैर लोडिंग और सत्यापन को संभालती है, जिसमें हर उपयोगकर्ता-सामने आने वाली त्रुटि स्ट्रिंग i18n.Translator के माध्यम से रूट की जाती है।

यह सब वास्तव में अपनाने योग्य बनाने वाला निर्णय यह है कि भारी नेटिव डिपेंडेंसी वैकल्पिक है। OpenCV बाइंडिंग्स को -tags vision के पीछे बिल्ड-टैग-गोटेड किया गया है, और डिफ़ॉल्ट बिल्ड स्टब्स के साथ आता है — ताकि पूरा मॉड्यूल किसी भी Go 1.25+ होस्ट पर बिना OpenCV टूलचेन के कंपाइल, टेस्ट और रन हो सके और नेटिव स्टैक तभी शामिल हो जब उपभोक्ता स्पष्ट रूप से इसके लिए ऑप्ट इन करे। यही वह चीज़ है जो VisionEngine को एक साधारण CI रनर में बिना किसी विशेष इमेज के आसानी से इंटीग्रेट करने देती है। पूरी तरह से डिक्पल्ड (संविधान के अनुसार CONST-051(B)), इसे उपभोक्ताओं — विशेष रूप से HelixQA — द्वारा समान-कोडबेस सबमॉड्यूल के रूप में शामिल किया जाता है, जिससे साक्ष्य-आधारित यूआई परीक्षण को वास्तविक दृष्टि मिलती है।

सामग्री

हमने इसे क्यों बनाया

सुलभता ट्री या चयनकर्ताओं पर निर्भर यूआई परीक्षण स्वचालन वह नहीं देख पाता जो उपयोगकर्ता वास्तव में देखता है। VisionEngine वास्तविक दृश्य समझ को जोड़ता है — तत्व पहचान, स्क्रीन तुलना और LLM-दृष्टि तर्क — साथ ही ऐप स्क्रीनों का एक नेविगेशन योग्य मानचित्र, ताकि स्वचालन यूआई को न केवल समझ सके बल्कि उसमें मार्ग भी तय कर सके।

यह क्यों गेम-चेंजर है

यह दो आमतौर पर असंगत दृष्टिकोणों — तेज़, नियतात्मक शास्त्रीय कंप्यूटर विज्ञान और लचीली, अर्थगर्भित LLM दृष्टि — को एक ही इंटरफ़ेस के पीछे लाता है, जिसमें फ़ॉलबैक शृंखला भी शामिल है। इससे उपभोक्ता को एक की सटीकता और दूसरे की तर्कशक्ति दोनों मिलती है, बिना किसी चुनाव के। और OpenCV को पूरी तरह वैकल्पिक रखकर, यह उस शक्ति पर लगने वाले सामान्य कर को हटा देता है: कोई भी Go प्रोजेक्ट वास्तविक यूआई बोध प्राप्त कर सकता है, बिना अपने बिल्ड में नेटिव विज्ञान टूलचेन को शामिल किए।

क्या है नवीन

- द्वैत बोध: शास्त्रीय सीवी (OpenCV/GoCV) के साथ-साथ बहु-प्रदाता LLM दृष्टि, जिसमें फ़ॉलबैक शृंखला भी शामिल है।
- नेविगेशन ग्राफ़ जिसमें BFS पथ-खोज और DOT/JSON/मेरमेड निर्यात की सुविधा।

- बिल्ड-टैग द्वारा नियंत्रित OpenCV, ताकि मॉड्यूल बिना नेटिव निर्भरताओं के भी बनाया और परीक्षित किया जा सके।
- पूरी तरह अलग-थलग, अंतरराष्ट्रीयकरण-संयोजित, समान कोडबेस वाला सबमॉड्यूल (जिसका उपयोग HelixQA द्वारा किया जाता है)।

चुनौतियाँ और समाधान

- भारी नेटिव निर्भरता की जटिलता: -tags vision गेटिंग और डिफॉल्ट स्टब्स द्वारा हल, ताकि OpenCV के बिना भी CI/होस्ट पर बिल्ड और टेस्ट हो सके।
- विज्ञान प्रदाताओं की अविश्वसनीयता: VisionProvider इंटरफ़ेस और FallbackChain कंपोजर द्वारा हल।
- जटिल ऐप प्रवाहों का मानचित्रण: निर्देशित नेविगेशन ग्राफ़, BFS पथ-खोज और बहु-प्रारूप निर्यात द्वारा हल।
- युग्मन: CONST-051(B) डिक्पलिंग और अंतरराष्ट्रीयकरण अनुवादक सीम द्वारा हल।

तकनीकी स्टैक (क्यों + कैसे)

- **Go (1.25+)** — मॉड्यूल का मूल और चारों परतें।
- **GoCV / OpenCV** — शास्त्रीय कंप्यूटर विज्ञान, बिल्ड-टैग द्वारा नियंत्रित।
- **LLM विज्ञान प्रदाता (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — एडेप्टर के माध्यम से बहुमॉडल यूआई तर्क।
- **ग्राफ़ एल्गोरिदम (BFS)** — नेविगेशन पथ-खोज।
- **DOT / JSON / मेरमेड निर्यातक** — नेविगेशन-ग्राफ़ विज़ुअलाइज़ेशन।
- **अंतरराष्ट्रीयकरण अनुवादक** — अलग-थलग उपयोगकर्ता-सामना वाले स्ट्रिंग्स।